

This piece outlines an investigation into a failure within the foreign exchange (Fx) data pipeline, specifically involving a file that arrived on a UK bank holiday but was not processed by either the scheduled or ad hoc DAG. The issue stemmed from the use of `datetime.now()` in the orchestration logic, which incorrectly referenced the current system date instead of the intended backdated value extracted from the filename. As a result, the pipeline skipped the curation step, leading to a risk of inaccurate rolling average calculations and potential downstream impact across the Finance platform.

To address the problem, I proposed code improvements, including updating the function to use the correct date from the filename. I also suggested enhancements to the debugging process: adding DAG descriptions to alert emails for quicker context, and improving the clarity of error messages to support faster resolution.

Beyond the immediate fix, I recommended creating a separate ad hoc DAG to handle weekend and bank holiday logic. This would simplify the central orchestration script, reduce unnecessary computation, and make the codebase easier to maintain. Additionally, I proposed a monitoring table in BigQuery to track file metadata, enabling early detection of missing files by users who do not have access to Composer or Storage buckets. Depending on team appetite and resources, dummy file creation could also be automated via a new DAG.

These suggestions received positive feedback and are being considered for future implementation to improve pipeline reliability and maintainability.

1. Introduction

The engineering team within the Finance platform is responsible for managing both finance reference data and master data. Reference data comprises standardised internal or external datasets used to classify or categorise other data. For example, Standard Industrial Classification (SIC) codes are numerical identifiers that categorise businesses based on their primary activities. Master data, on the other hand, refers to core business entities that provide

essential context for transactions, such as supplier information. This portfolio piece addresses an issue related to foreign currency exchange rates (Fx) reference data.

The current data architecture operates as follows. Data files are received daily and deposited into a designated inbound bucket within Google Cloud Storage. Cloud Composer DAGs orchestrate a multi-stage pipeline that includes:

- Reading and validating inbound files;
- Ingesting data into the Raw BigQuery dataset;
- Staging data into intermediate BigQuery tables using DBT;
- Curating data into finalised tables;
- Populating consumption-ready tables;
- Generating output files from consumption tables;
- Exporting the files to an outbound Google Cloud Storage bucket.

The export process produces three CSV files, which are transmitted to relevant APIs every day, including weekends and bank holidays. These files are critical for calculating rolling averages of Fx and exchange rates. Timely receipt and accurate date alignment of these files are essential to ensure the integrity of these calculations.

Although files arrive daily, ingestion only occurs on weekdays, excluding bank holidays and weekends. Files received during weekends or holidays are expected to be processed on the next available working day. If raw ingestion fails, the pipeline stops, an alert email is triggered (see Fig. 1), and the failed file is relocated to an error folder for further investigation.

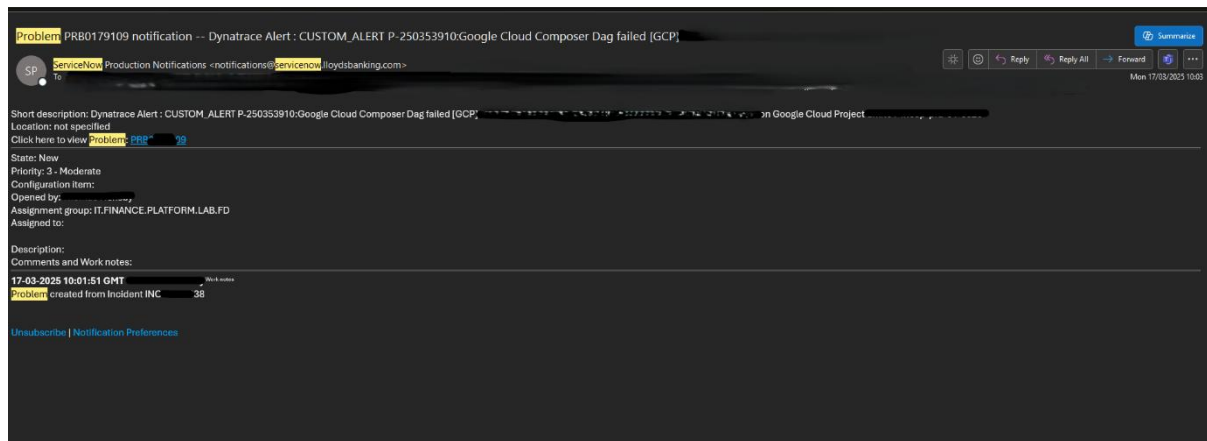


Figure 1 – Alert email

In foreign exchange (Fx) data processing, standard orchestration relies on the current date to determine whether to run ingestion or curation workflows. However, when a file is missed and needs to be processed retrospectively, this approach is insufficient. To address this, an ad hoc sensor DAG has been developed, allowing manual triggering based on a specified backdated file. This DAG uses a specially named file (ADHOC_YYYYMMDD.csv), placed in a designated bucket, to determine the intended processing date, independent of the current date (see Fig. 2). Depending on whether the specified date is a working day or a non-working day (weekend or bank holiday), the DAG will trigger either the ingestion or curation stage accordingly. Importantly, this ad hoc mechanism operates separately from the routine scheduled DAG and does not interfere with regular daily processing.

The problem occurred when the file arrived on a bank holiday and the ad hoc DAG failed to process it for that date. The risk of data not being fit for purpose arose as a result, as rolling averages calculations would be incorrect without the missing file. Moreover, as this is reference data, the incorrect calculations will be cascaded to all other teams in the Finance platform, increasing the risk of using invalid data. Once the Product Owner was notified about the issue, the details were shared with the respective risk owners and data consumers.

Section 2 outlines the methodology and steps used to investigate the problem.

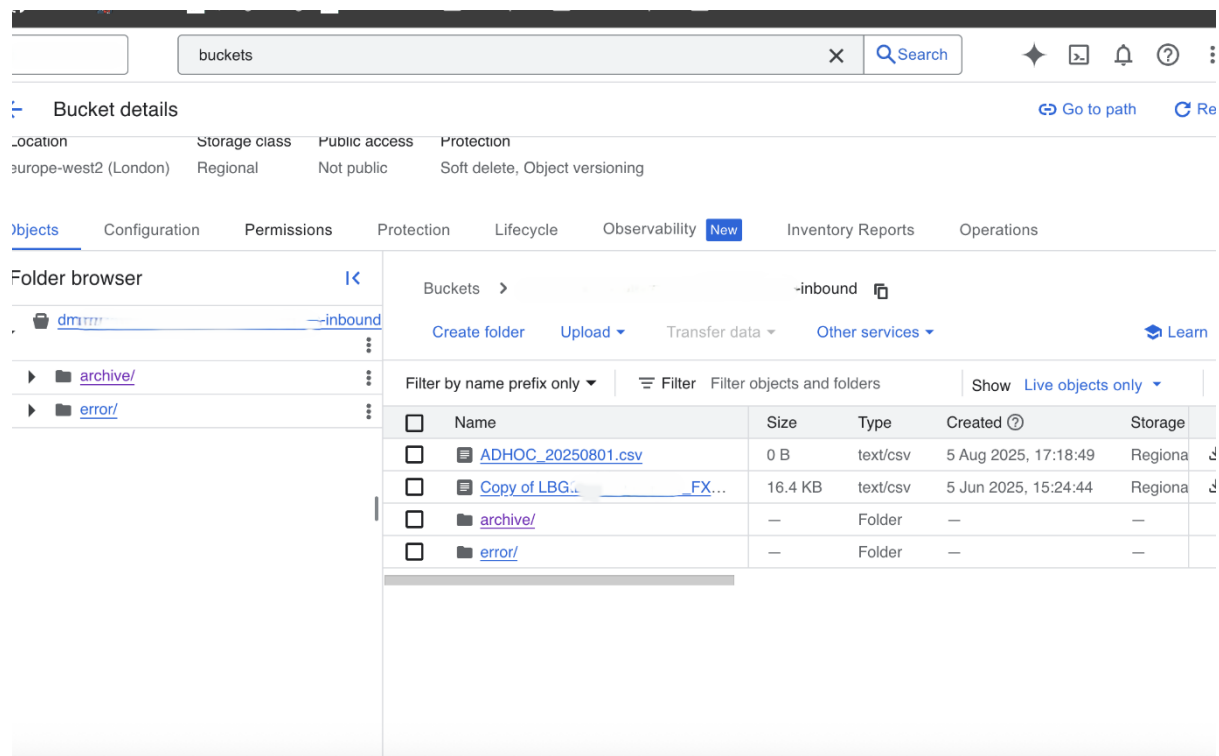


Figure 2 – GCS bucket with the ad hoc file

2. Methodology & Findings

After the team received the alert email, the investigation began. The email mentioned the DAG name, GCP project, and the URL to the error message. By checking the error message and the error folder, I identified that the bank holiday file was processed neither by the relevant DAG nor by the ad hoc DAG. There is a central DAG that is used to orchestrate all the other DAGs for various scenarios. As the architecture is rather complex and covers multiple use cases, the central DAG is a lengthy piece of code and is prone to mistakes. By analysing the code, and the `is_weekend_or_holiday` function in particular (as shown in Fig. 3), I found out the cause of the problem.

The issue with the bank holiday file not being processed appears to stem from the use of `datetime.now()` in the `is_weekend_or_holiday` and `update_config` functions. In both functions, the code uses `datetime.now()` to determine the current date. This is problematic in an ad hoc run, where the processing should be based on a backdated value (from the filename, such as

ADHOC_YYYYMMDD.csv), rather than the current system date. For ad hoc runs, the logic should use the date extracted from the filename (via `extract_date_from_filename`) to determine whether it is a weekend or bank holiday. However, `is_weekend_or_holiday` still checks `datetime.now().date()` against the holiday list, which leads to incorrect branching and skips the intended curation step. If the current date is a weekday but the ad hoc date is a weekend or holiday, the DAG incorrectly proceeds with ingestion instead of curation. This causes the bank holiday file to be ignored or misprocessed. I suggested updating the `is_weekend_or_holiday` function to use the ad hoc run date from the DAG configuration instead of `datetime.now()`.

```
# Below functions to check weekday, weekend and UK bank holiday from config.
def is_weekend_or_holiday(**context):
    config_data = context['ti'].xcom_pull(task_ids='read_config_file', key='config_data')
    client = storage.Client()
    adhoc_bucket = config_data['COMMON']['ADHOC_GCS_BUCKET']
    bucket = client.bucket(adhoc_bucket)
    blob = bucket.get_blob('holiday_calendar/uk_holiday_calendar.csv')
    content = blob.download_as_string().decode('utf-8')
    df = pd.read_csv(StringIO(content))
    holidays_data = df['holiday_date'].dropna().tolist()

    print(f"uk_bank_holidays: \{holidays_data\}")
    now = datetime.now()
    print(now)
    if now.weekday() >= 5 or str(now.date()) in holidays_data:
        print("update_config_file")
        return 'update_config_file'
    else:\
        print("fxrates_trans_sensor")
        return 'fxrates_trans_sensor'
```

Figure 3 – The function to identify weekends or bank holidays

To summarise, I followed these steps to identify the root cause of the DAG failure. I began by reviewing the error messages in the Google Cloud Composer console to pinpoint where the failure occurred. Next, I examined the successful stages to understand the overall pipeline flow. I then located and analysed the relevant file in the error folder, checking its name, format, and content. After that, I reviewed the code to identify the section responsible for handling bank holiday scenarios. Finally, I analysed the logic within the function and

discovered that it incorrectly used the current system date instead of the intended backdated value. I shared my findings with the team and suggested code improvements. My ideas received positive feedback and were implemented in the same sprint. Deployment and testing were completed just in time for the August bank holiday, and the file was processed successfully.

3. Recommendations

In addition to recommending code changes to resolve the issue, I proposed several enhancements aimed at improving the debugging process and overall pipeline efficiency. Firstly, I suggested that the alert email should include a brief description of the DAG alongside its name. This would allow the team to quickly understand the context and potential impact of the failure without needing to inspect the code or possess detailed knowledge of the orchestration logic. Such contextual information would support faster risk assessment and more informed decision-making.

Secondly, although most error messages provided sufficient detail to diagnose issues, some exceptions lacked clarity and specificity (see Fig. 4). In these cases, more explicit and informative error messages would significantly aid troubleshooting efforts. Clearer messaging would reduce the time spent interpreting ambiguous logs and minimise the risk of misdiagnosis, particularly in complex scenarios involving multiple DAGs and dependencies.

manual__2025-08-01T12:24:04.357037+00:00		
Logs for DAG run: manual__2025-08-01T12:24:04.357037+00:00		
Severity	Timestamp	Summary
> i	2025-08-01 12:33:19.292 UTC	Executing callback at index 0: custom_failure_callback
✓ !!	2025-08-01 12:33:19.295 UTC	Notification: DAG Failure Detected! Source System Name: FX DAG ID: fx_orchestrator Task ID: fx_rates_consumption_trigger Execution Date: 2025-08-01 12:24:04.357037+00:00 Error: fx_rates_consumption_gkepodstart failed with failed states failed
Notification: DAG Failure Detected! Source System Name: FX DAG ID: fx_orchestrator Task ID: fx_rates_consumption_trigger Execution Date: 2025-08-01 12:24:04.357037+00:00 Error: fx_rates_consumption_gkepodstart failed with failed states failed		
Investigate Open in Logs Explorer		

Figure 4 – DAG logs with unclear error message

Moreover, I recommended creating a dedicated ad hoc DAG to handle weekend and bank holiday retry logic, moving this functionality out of the central orchestration script. This change would simplify the codebase, which currently exceeds 300 lines and covers a wide range of scenarios. Isolating the retry logic into a separate DAG would not only make debugging easier but also improve maintainability and reduce the computational overhead associated with executing unnecessary code.

To further support monitoring and transparency, I proposed the creation of a dedicated BigQuery table to track file names, extracted dates, and processing timestamps. This would enable engineers and data professionals, particularly those without access to the buckets or Composer environment, to identify missing files early and take corrective action. Depending on the team's appetite for automation and available resources, the placement of ad hoc dummy files into the bucket could also be automated. Currently, an engineer manually creates an empty file with the same date in the filename as the failed file. If the monitoring table were implemented, a new DAG could be introduced to automatically generate the required dummy file based on the missing date.

I received positive feedback on these improvement suggestions, and they are being considered for future implementation to enhance the robustness and maintainability of the FX data pipeline.

4. Conclusion

This investigation highlighted several important areas for improvement in the FX data pipeline. One key learning was the need to ensure that orchestration logic reflects the actual business context, particularly when handling exceptions such as missed files on weekends or bank holidays. The current setup, while functional for routine processing, lacks flexibility and transparency in ad hoc scenarios.

The experience also emphasised the value of clearer communication and better tooling. Enhancing alert messages with more descriptive context and improving the clarity of error outputs would help teams respond more quickly and accurately to issues. Simplifying the orchestration by separating specific logic into dedicated workflows would make the codebase easier to maintain and reduce unnecessary processing.

Another key takeaway from this experience was the importance of cross-collaboration with another engineering team. I discovered that, once the development work was completed, it was handed over to a different team responsible for monitoring the process and resolving minor issues. They use a runbook created by the development team, which contains common errors, troubleshooting guidance, and mitigation steps. The runbook was updated accordingly after the fix, and the monitoring team was notified of the changes.

Looking ahead, introducing a monitoring mechanism to track file status and processing history would provide greater visibility across teams, especially for those without direct access to orchestration tools. Automating parts of the recovery process, such as dummy file creation, could further reduce manual effort and improve consistency.

Overall, the investigation not only resolved the immediate issue but also opened up opportunities to strengthen the pipeline's resilience, maintainability, and usability. These recommendations are practical, scalable, and aligned with the broader goals of improving data reliability across the Finance platform.